Stock Market Analytics & Forecasting

Project Design and Technical Documentation

Project type	Full-stack machine-learning web application
Primary user input	Stock ticker and forecast period
Forecast periods	7, 15, 30, 60, or 90 trading days
Persistence	MySQL through SQLAlchemy 2.x
Interface	FastAPI backend with HTML, CSS, JavaScript, and Plotly
Documentation date	2026-09-25

Purpose

This application collects historical market data, prepares it for time-series modelling, trains three complementary forecasting models, evaluates them using chronological validation, combines their outputs using validation-derived weights, and presents the result through an interactive analytics dashboard.

Disclaimer

The application provides statistical/model-based forecasts for educational and analytical purposes. Forecasts are uncertain and should not be treated as guaranteed future prices or financial advice.

1. Executive Overview

The user experience is intentionally simple: the user enters a ticker such as AAPL and selects a forecast horizon. Historical period selection, model selection, technical indicators, model weights, and ensemble logic are handled automatically by the application.

Capability	Design decision
Data acquisition	Yahoo Finance through yfinance, with a ten-year daily history request.
Caching	Historical prices are stored in MySQL and reused when sufficiently recent.
Preprocessing	Dates are normalized, duplicates removed, missing values filled, and numeric values validate
Modelling	ARIMA, Random Forest, and LSTM provide complementary statistical, tree-based, and neura
Evaluation	Chronological validation prevents random shuffling and future leakage.
Presentation	Plotly charts, range controls, pan/zoom, forecast boundary markers, and readable tables.

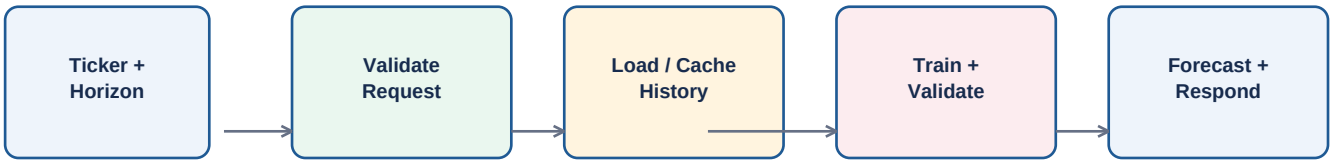
2. High-Level Architecture

System architecture

The browser communicates with FastAPI using HTTP. The service layer coordinates database access, data retrieval, preprocessing, model training, evaluation, and response construction. MySQL stores the reusable historical data and forecast results.

3. End-to-End Request Flow

Forecast request lifecycle



1. The frontend validates the basic input and sends **GET /api/forecast**. 2. FastAPI validates the ticker and supported horizon. 3. The data service checks MySQL before using yfinance. 4. The forecast service creates features, fits models, performs chronological validation, calculates metrics and weights, and generates the requested future path. 5. The response is rendered into summary cards, charts, model tables, and the forecast table.

4. Backend Modules

Module	Responsibility
app/main.py	Creates the FastAPI application, registers CORS/static routes, initializes the database, expos
app/core/config.py	Loads environment configuration, database URL, frontend path, artifact path, logging level, a
app/core/database.py	Creates the SQLAlchemy engine/session factory, exposes request-scoped sessions, and initi
app/models/	Defines relational SQLAlchemy entities and relationships for stocks, prices, forecast runs, pre
app/schemas/	Defines Pydantic validation for tickers, forecast periods, response records, model metrics, an
app/services/data_service.py	Downloads and cleans yfinance data, handles multi-level columns, retries provider requests,
app/services/forecast_service.py	Coordinates feature construction, chronological validation, model training, recursive multi-step
app/services/ensemble_service.py	Normalizes validation-derived model weights and combines model predictions.
app/ml/	Contains ARIMA, Random Forest, LSTM, feature engineering, and metric calculation logic.
app/utils/	Contains ticker validation, forecast-period validation, numeric conversion, and small reusable

5. Data and Database Design

Persistence flow

Table	Purpose and key constraints
stocks	One row per ticker. Unique ticker, optional company metadata, created and updated timestam
stock_prices	Daily OHLCV history. Foreign key to stocks, unique stock/date constraint, and stock/date inde
forecast_runs	One record per forecast request, including horizon, current price, predicted price, and change
forecast_predictions	One row per future date with ARIMA, Random Forest, LSTM, and ensemble predictions.
model_metrics	Validation MAE, RMSE, and MAPE for each model and forecast run.
model_weights	Validation-derived ensemble contribution for each model and forecast run.

The database is production-oriented MySQL rather than an in-memory or SQLite substitute. Tables are initialized from SQLAlchemy metadata at application startup, and the equivalent DDL is also provided in **sql/schema.sql**.

6. Data Quality and Caching

- Ticker values are normalized to uppercase and restricted to safe ticker characters.
- Yahoo Finance requests use a ten-year daily interval, bounded retries, and a documented history fallback.
- Duplicate dates are removed, time indexes are normalized, NaN values are filled where appropriate, and rows without a close price are discarded.
- A recent cache with sufficient history is reused; otherwise the service downloads and upserts fresh data.

7. Feature Engineering

Features are calculated using information available at or before each prediction timestamp. The feature matrix is sorted chronologically and rows with unavailable lagged values are removed before model fitting.

Feature group	Features
Price lags	lag_1, lag_2, lag_3, lag_5, lag_7, lag_14, lag_30
Moving averages	SMA_20, SMA_50, SMA_200, EMA_20
Return features	daily_return, return_lag_1, return_lag_5, return_lag_10, return_lag_20
Momentum	20-session price momentum and distance from SMA 20, 50, and 200
Risk/volatility	20-session rolling volatility and 60-session volatility

8. Machine-Learning Pipeline

Model training and ensemble flow

ARIMA: Fits log prices with first differencing, a small candidate-order search, AIC selection, and a linear drift term. Forecasts are exponentiated back to price units.

Random Forest: Predicts next-session returns from lagged price, return, momentum, moving-average, and volatility features. Recursive forecasts are converted back to prices, with bounded return handling for high-volatility conditions.

LSTM: Predicts return sequences using a lightweight recurrent network. A StandardScaler is fitted on training returns only, and predictions are inverse-transformed before being converted into prices.

Ensemble: Each model is evaluated on a chronological holdout using the same multi-step strategy used in production. Lower validation error receives more weight, with a small equal-weight stabilization to avoid total dependence on one model.

9. Validation, Metrics, and Leakage Controls

- Time-series observations are never randomly shuffled.
- The selected forecast horizon is used for validation, so a 7-day request is evaluated against a 7-day holdout and a 90-day request against a 90-day holdout.
- Feature rows are split chronologically; future validation targets are not used to fit the models.
- The LSTM scaler is fitted on training returns only.
- Recursive Random Forest and LSTM forecasts use their own prior predictions rather than future actual prices.
- MAE, RMSE, and MAPE are calculated on held-out validation observations.

Metric	Meaning
MAE	Average absolute forecast error in price units.
RMSE	Square-root average squared error; penalizes larger misses more heavily.
MAPE	Average absolute percentage error, reported as a percentage.

10. Frontend and User Experience

Frontend area	Behavior
Input controls	Ticker input, custom dark/light-compatible forecast-period menu, and disabled state during ge
Summary cards	Current price, forecast price, projected change, and selected horizon.
Historical chart	Plotly close-price chart with 3M, 5M, 1Y, 2Y, and All range controls plus range slider.
Forecast chart	Historical line joined to the ensemble path; individual model paths are available through the le
Interaction	Hover labels, pan mode, scroll zoom, Y-axis panning, reset/autoscale controls, and forecast b
Tables	Model metrics, weights, daily forecasts, and recent forecast records.
Responsive layout	Cards, tables, controls, and charts adapt to narrower screens.

11. API and Runtime Flow

Endpoint	Purpose
GET /	Serves the dashboard HTML entry point.
GET /health	Checks application availability and reports database connectivity.
GET /api/stocks	Lists known stocks stored in MySQL.
GET /api/forecast?ticker=AAPL&forecast_days=30	Validates the request, loads history, trains models, generates the ensemble, ar
GET /api/forecast-history	Provides the recent forecast-history collection used by the dashboard.

12. Project Structure

```
stock-forecasting-ml/
■■■ app/
■ ■■■ main.py
■ ■■■ core/ configuration and database
■ ■■■ models/ SQLAlchemy persistence models
■ ■■■ schemas/ Pydantic request/response schemas
■ ■■■ services/ data, preprocessing, forecasting, ensemble services
■ ■■■ ml/ ARIMA, Random Forest, LSTM, metrics, features
■ ■■■ utils/ validation and helper functions
■■■ frontend/ HTML, CSS, JavaScript, Plotly
■■■ tests/ pytest tests
■■■ sql/schema.sql MySQL DDL
■■■ artifacts/ optional model/scaler storage
■■■ requirements.txt
■■■ .env.example
■■■ README.md
■■■ run.py
```

13. Installation and Operation

Create the database, configure the connection string in **.env**, install requirements, and start the application from the project root.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python run.py
```

Open **http://localhost:8000**. The FastAPI process serves both the backend and frontend. The application initializes database tables on startup when MySQL is available.

14. Testing and Verification

Test area	Coverage
API	Health endpoint, ticker validation, forecast-period validation, and schema normalization.
Feature engineering	Lag generation, technical indicators, and feature matrix construction.
Preprocessing	Missing-value handling, duplicate-date removal, sorting, and normalization.
Ensemble	Validation-based weights, fallback behavior, and weighted combination.
Runtime checks	Python compilation, FastAPI import/startup, static asset serving, and end-to-end cached forec

15. Error Handling and Reliability

- Detailed errors are logged on the server without exposing credentials or raw stack traces to users.
- Invalid tickers and unsupported horizons are rejected at the API boundary.
- Yahoo Finance failures return a user-safe message and do not create fabricated market data.
- MySQL connection problems are visible through the health endpoint and startup logs.
- Model-training failures are converted into controlled API responses.

16. Limitations and Future Improvements

- Point forecasts cannot reliably reproduce the random day-to-day noise of financial markets.
- External market, sector, macroeconomic, and news variables are not currently included.
- A future version could add rolling-origin cross-validation across several historical regimes and an untouched final test period.
- Prediction intervals, quantile forecasts, and calibrated uncertainty bands would improve risk communication.
- Forecast-run persistence and historical run loading can be expanded for a richer audit trail.

Final note

The design prioritizes transparent data handling, chronological evaluation, understandable modules, and a dashboard that clearly separates historical observations from model-based future forecasts.